

Fiche résumé : le pattern Décorateur

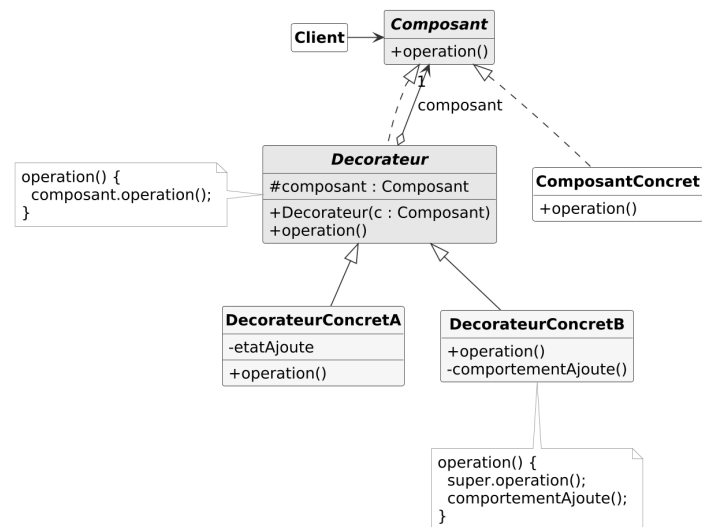
Decorator, aussi appelé Wrapper | Pattern de structure du GoF, portée objet

Intention métier

Ajouter dynamiquement de nouvelles responsabilités à un objet en l'enveloppant dans un autre objet qui a la même interface. C'est une alternative plus souple que l'héritage pour étendre un comportement (d'après Gamma et al., 1994).

Côté métier, on y pense dès qu'un produit ou un service de base peut recevoir des **options cumulables**, choisies au moment de l'utilisation, et que la liste de ces options évolue souvent : suppléments d'une commande, options d'un billet ou d'un forfait, bonus d'un personnage, traitements successifs sur un flux de données.

Diagramme de classes



Le Decorateur **implémente** Composant et **contient** un Composant : c'est ce double lien qui permet d'empiler les décorateurs. Chaque appel traverse les couches de l'extérieur vers l'intérieur.

Classes participantes

Classe participante	Rôle
Composant (Component)	Interface commune aux objets de base et aux décorateurs. Seul type connu du client.
ComposantConcret (ConcreteComponent)	Objet de départ que l'on veut enrichir. Il ignore qu'il peut être décoré.
Decorateur (Decorator)	Classe abstraite qui implémente Composant et garde une référence vers un Composant. Par défaut, elle délègue.
DecorateurConcret (ConcreteDecorator)	Ajoute un état ou un comportement avant ou après la délégation. Un par option.
Client	Construit la pile de décorateurs et manipule le résultat comme un simple Composant.

Exemple dans un contexte métier : la borne de commande d'un snack

Un tacos de base (poulet à 7,50 €, végété à 7,00 €) peut recevoir des suppléments cumulables : cheddar (+1,00 €), bacon (+1,50 €), gratiné (+0,80 €). Le gérant ajoute régulièrement de nouveaux suppléments. Une classe par combinaison ferait exploser le nombre de classes, et des booléens dans Tacos obligeraient à modifier cette classe à chaque nouveauté.

```

public interface Tacos { String description(); double prix(); }

public abstract class Supplement implements Tacos {
    protected final Tacos tacos;
    protected Supplement(Tacos tacos) { this.tacos = tacos; }
}

public class Cheddar extends Supplement {
    public Cheddar(Tacos tacos) { super(tacos); }
    public String description() { return tacos.description() + " + cheddar"; }
    public double prix() { return tacos.prix() + 1.00; }
}

Tacos commande = new Gratine(new Cheddar(new Cheddar(new TacosPoulet())));
commande.prix(); // 7.50 + 1.00 + 1.00 + 0.80 = 10.30

```

Correspondance : Tacos = Composant, TacosPoulet et TacosVege = ComposantConcret, Supplement = Decorateur, Cheddar, Bacon et Gratine = DecorateurConcret.

SOLIDité du pattern

Principe	En quoi le Décorateur le respecte
S , responsabilité unique	Chaque décorateur concret ne gère qu'une option. Changer le prix du bacon ne touche que la classe Bacon.
O , ouvert / fermé	Ajouter un supplément, c'est écrire une nouvelle classe. Tacos, TacosPoulet et les autres suppléments ne sont pas modifiés.
L , substitution de Liskov	Un tacos décoré s'utilise partout où un Tacos est attendu, à condition que chaque décorateur respecte le contrat (par exemple, un prix jamais négatif).
I , ségrégation des interfaces	Chaque décorateur doit implémenter toutes les méthodes de Composant : une interface courte garde le pattern léger.
D , inversion des dépendances	Supplement et le client dépendent de l'abstraction Tacos, jamais d'une classe concrète. On peut décorer n'importe quelle implémentation.

Résultat : un code plus maintenable. Une nouvelle option ne demande ni de rouvrir ni de retester le code existant, et chaque option se teste isolément. C'est l'application directe du conseil du GoF : préférer la composition à l'héritage.

Limites

L'ordre des couches peut changer le résultat (réduction en pourcentage avant ou après un supplément). Beaucoup de petites classes qui se ressemblent. Un objet décoré n'a plus la même identité que l'original (instanceof, equals). Retirer une couche au milieu de la pile n'est pas prévu. La construction imbriquée est verbeuse, on l'accompagne souvent d'une fabrique ou d'un builder. Si Composant a beaucoup de méthodes, chaque décorateur doit toutes les déléguer.

Dans le JDK

Les flux de java.io : new BufferedReader(new FileReader("f.txt")), new DataInputStream(new BufferedInputStream(...)). Ici, InputStream joue le rôle de Composant et FilterInputStream celui de Decorateur. Autres exemples : Collections.unmodifiableList(liste), Collections.synchronizedList(liste).

Références

Gamma E., Helm R., Johnson R., Vlissides J., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

Freeman E., Freeman E., Sierra K., Bates B., *Head First Design Patterns*, O'Reilly, 2004, chapitre 3.

Martin R. C., *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2002.

Refactoring.Guru, Décorateur : <https://refactoring.guru/fr/design-patterns/decorator>